

Jarvis — Full Architecture & Implementation Guide



Implementation plan for Haris — Read this document completely before writing a single line of code. Every design decision here is intentional — understand the *why*, not just the *what*.

0. Access & Infrastructure (Set This Up First)

Before Phase 0 — get accounts and access sorted:

- **Code repo:** Create a **private GitHub repository** under the **Nexrova Technologies** organization. Ask **Stuti** to add Haris to the Nexrova Technologies org and grant him **write access** (to the org and the repo). Never commit `.env` or any secrets.
- **LLM:** Use **Microsoft Azure OpenAI** with a strong, suitable model — **GPT-4o** (or the strongest model available in your Azure region). Ask **Stuti** for the Azure OpenAI **API key**, **endpoint**, and **deployment name**.
- **Deployment:** Deploy on **Render** (same platform as the existing bot).
- **Database:** Use **Supabase** — the same project as the existing bot.

1. What Jarvis Is

Jarvis is a Discord bot that acts as Nexrova's operating brain. It does two things:

1. **Tracks everything** — tasks, work sessions, blockers, standups, progress, slippage
2. **Understands what it's tracking** — uses a large language model (Microsoft Azure OpenAI) to turn raw data into actionable intelligence

The flywheel: the more the team uses it, the richer the data, the better the insights. Week 1 it's useful. Week 8 you can't run the team without it.

This replaces the existing clock-in/out bot. All existing functionality migrates in, and new features layer on top.

2. Tech Stack

Component	Tool
Bot framework	discord.py 2.x
Database	Supabase (PostgreSQL) — same project as existing bot
AI / LLM	Microsoft Azure OpenAI — GPT-4o (use the strongest model available in your Azure region). Ask Stuti for the API key.
Scheduling	APScheduler 3.x
Language	Python 3.11+
Deployment	Render (same platform as existing bot)

3. Project Structure

```
jarvis/  
├─ main.py           # Entry point – loads all cog  
s, starts bot  
├─ config.py        # All env vars and constants i  
n one place  
├─ database.py      # Supabase client + reusable D  
B helpers  
├─ ai_engine.py     # Claude API calls + context c  
ompilation  
├─ scheduler.py     # APScheduler jobs (briefing,  
reminders, narrative)  
├─ utils.py         # Shared utilities (time forma  
tting, embeds, checks)  
├─ requirements.txt  
├─ .env             # NEVER commit this file  
├─  
├─ cogs/  
│   └─ __init__.py
```

```

|   |— clock.py                # /clockin, /clockout, /mystat
s (migrated)
|   |— stats.py                # /teamstatus, /serverreport
(migrated)
|   |— tasks.py                # /task-create, /task-done, /p
ickup, etc.
|   |— modules.py              # /module-create, /module-stat
us
|   |— standups.py             # /standup + DM reminder flow
|   |— blockers.py             # /blocker, /blocker-resolve
|   |— intelligence.py          # /profile, /growth, /delivery
-check, @jarvis listener
|   |— demo.py                 # /demo-ready, /demo-update
|   |— admin.py                # /register, /context-add, /vi
ewlog, /resetmonth
|
└— prompts/
    |— morning_briefing.txt
    |— weekly_narrative.txt
    |— person_profile.txt
    |— delivery_prediction.txt
    └— ops_brain.txt

```

4. Database Schema

Run this SQL in Supabase → SQL Editor in the order shown. Existing tables (`active_sessions`, `completed_sessions`, `archived_sessions`) stay untouched.

4.1 Team Members

```

CREATE TABLE team_members (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid
()),
discord_user_id    TEXT UNIQUE NOT NULL,
username           TEXT NOT NULL,

```

```

display_name    TEXT,
role            TEXT NOT NULL DEFAULT 'intern',
-- 'intern' | 'founder' | 'admin'
is_active       BOOLEAN DEFAULT TRUE,
joined_at       TIMESTAMPTZ DEFAULT NOW()
);

```

4.2 Modules (Feature Groups / Project Areas)

```

CREATE TABLE modules (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name          TEXT NOT NULL,
  description    TEXT,
  client_name    TEXT,                -- e.g. 'Kolam'
  deadline       TIMESTAMPTZ,
  status         TEXT DEFAULT 'active',
-- 'active' | 'completed' | 'paused'
  created_by     TEXT,                -- discord_user_id
  created_at     TIMESTAMPTZ DEFAULT NOW()
);

```

4.3 Tasks

```

CREATE TABLE tasks (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title         TEXT NOT NULL,
  description    TEXT,
  module_id      UUID REFERENCES modules(id) ON DELETE SET NULL,
  assignee_id    TEXT,                -- discord_user_id, NULL = unassigned
  creator_id     TEXT NOT NULL, -- discord_user_id
  status         TEXT DEFAULT 'todo',
-- 'todo' | 'in_progress' | 'in_review' | 'done' | 'blocked' | 'cancelled'
);

```

```

priority          TEXT DEFAULT 'medium',
-- 'low' | 'medium' | 'high' | 'critical'
estimated_hours   FLOAT,
due_date          TIMESTAMPTZ,
created_at        TIMESTAMPTZ DEFAULT NOW(),
started_at        TIMESTAMPTZ,
completed_at      TIMESTAMPTZ,
slipped           BOOLEAN DEFAULT FALSE,
slip_count        INTEGER DEFAULT 0 -- how many times due da
te was pushed
);

CREATE INDEX idx_tasks_assignee ON tasks(assignee_id);
CREATE INDEX idx_tasks_status ON tasks(status);
CREATE INDEX idx_tasks_module ON tasks(module_id);

```

4.4 Task Updates (Progress Notes)

```

CREATE TABLE task_updates (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  task_id     UUID NOT NULL REFERENCES tasks(id) ON DELETE C
ASCADE,
  user_id     TEXT NOT NULL,
  update_text TEXT NOT NULL,
  update_type TEXT DEFAULT 'progress',
-- 'progress' | 'blocker' | 'completion' | 'pickup' | 'hand
off'
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

```

4.5 Standups

```

CREATE TABLE standups (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     TEXT NOT NULL,

```

```

date          DATE NOT NULL,
yesterday     TEXT,           -- what did you work on?
today         TEXT,           -- what's your focus today?
blockers      TEXT,           -- any blockers or risks?
submitted_at  TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(user_id, date)         -- one standup per person per day
);

CREATE INDEX idx_standups_date ON standups(date);

```

4.6 Blockers

```

CREATE TABLE blockers (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  reporter_id       TEXT NOT NULL,      -- who filed it
  task_id           UUID REFERENCES tasks(id) ON DELETE SET NULL,
  description       TEXT NOT NULL,
  blocking_person_id TEXT,              -- who's blocking them (if a dependency)
  status            TEXT DEFAULT 'open',
  -- 'open' | 'resolved' | 'stale'
  created_at        TIMESTAMPTZ DEFAULT NOW(),
  resolved_at       TIMESTAMPTZ,
  resolved_by       TEXT,
  resolution_note    TEXT,
  hours_open        FLOAT               -- filled in when resolved
);

```

4.7 Slip Reasons

```
CREATE TABLE slip_reasons (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid
  (),
  task_id     UUID NOT NULL REFERENCES tasks(id) ON DEL
ETE CASCADE,
  user_id     TEXT NOT NULL,
  original_due_date TIMESTAMPTZ,
  new_due_date TIMESTAMPTZ,
  reason_category TEXT,
  -- 'dependency' | 'underestimate' | 'priority_switch' | 'ex
ternal'
  reason_text TEXT,
  recorded_at TIMESTAMPTZ DEFAULT NOW()
);
```

4.8 Context Log (Jarvis's Accumulating Memory)

This is the most important table. **Every meaningful event in the system gets written here as a plain English sentence.** This is what feeds the AI.

```
CREATE TABLE context_log (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_type  TEXT NOT NULL,
  -- 'task_create' | 'task_assign' | 'task_start' | 'task_com
plete'
  -- | 'task_slip' | 'task_pickup' | 'blocker_open' | 'blocke
r_resolve'
  -- | 'standup' | 'session_end' | 'manual' | 'demo_update'
  user_id     TEXT,
  task_id     UUID,
  module_id   UUID,
  content     TEXT NOT NULL,      -- human-readable event summa
ry (sent to AI)
  metadata    JSONB,             -- structured data for analyt
ics queries
```

```

    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- These indexes are critical – context compilation queries hit this table hard
CREATE INDEX idx_context_log_created ON context_log(created_at DESC);
CREATE INDEX idx_context_log_user ON context_log(user_id, created_at DESC);
CREATE INDEX idx_context_log_type ON context_log(event_type, created_at DESC);

```

Example rows:

```

event_type: "task_complete"
content:     "Adhi completed 'API endpoint build' in 4.2h (estimated 3h, 40% over)."
```

```

metadata:    {"estimated_hours": 3, "actual_hours": 4.2, "slipped": false, "module": "Dashboard"}
```

```

event_type: "blocker_open"
content:     "Vedanth reported a blocker on 'DB schema review' – waiting on Adhi's input. Task set to blocked."
```

```

metadata:    {"blocker_id": "...", "blocking_person": "adhi", "task_id": "..."}

```

```

event_type: "standup"
content:     "Stuti standup: Yesterday finished dashboard CSS. Today working on filter logic. No blockers."
```

```

metadata:    {"date": "2025-06-28"}

```

4.9 Jarvis Snapshots (Compressed Historical Memory)

After 4+ weeks, context_log gets large. Weekly snapshots compress old history so the AI context window stays manageable.


```
CREATE TABLE jarvis_snapshots (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  snapshot_type TEXT NOT NULL,
  -- 'weekly_team' | 'person_profile' | 'module_status'
  reference_id TEXT,          -- user_id or module name
  week_start   DATE,
  content      TEXT NOT NULL, -- AI-generated or structured
  summary
  raw_data     JSONB,          -- the metrics that went into
  this snapshot
  created_at    TIMESTAMPTZ DEFAULT NOW()
);
```

4.10 Demo Features

```
CREATE TABLE demo_features (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid
  (),
  name        TEXT NOT NULL,
  description  TEXT,
  module_id    UUID REFERENCES modules(id) ON DELETE SET
  NULL,
  status      TEXT DEFAULT 'not_ready',
  -- 'ready' | 'partial' | 'not_ready'
  status_note  TEXT,          -- e.g. "80% done, Stuti fini
  shing today"
  last_updated_by TEXT,
  last_updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

4.11 Manual Context

Jeevan and Subrat can directly inject things Jarvis should know — client decisions, risks, priorities — that don't come from tasks or sessions.

```
CREATE TABLE manual_context (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  added_by    TEXT NOT NULL,
  content     TEXT NOT NULL,
  context_type TEXT DEFAULT 'general',
  -- 'client' | 'decision' | 'risk' | 'general'
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

5. All Commands (Full List)

Permission levels

- **Anyone** — all team members
- **Founder** — Jeevan + Subrat (set via `FOUNDER_IDS` env var)
- **Admin** — destructive or sensitive actions (set via `ADMIN_IDS` env var)

`cogs/clock.py` — Time Tracking (Migrated)

Command	Permission	Description
<code>/clockin</code>	Anyone	Start work session
<code>/clockout</code>	Anyone	End session + work summary modal
<code>/mystats</code>	Anyone	Your quick work stats
<code>/myreport</code>	Anyone	Full personal report with monthly breakdown
<code>/lifetimehours</code>	Anyone	All-time hours across archived sessions
<code>/editsessions</code>	Anyone	View recent sessions for editing
<code>/edittime</code>	Anyone	Edit a past session clock-in or clock-out

Changes from existing: Migrate into cog structure, no behavior changes. Add one new thing: after the clockout modal saves, call `jarvis_ai.log_event()` with the session summary.

`cogs/stats.py` — Team Stats (Migrated)

Command	Permission	Description
<code>/teamstatus</code>	Anyone	Who's currently clocked in
<code>/serverreport</code>	Anyone	Full team hours with bar chart

No changes from existing. Just migrated to cog.

`cogs/tasks.py` — Task Management

Command	Permission	Description
<code>/task-create</code>	Anyone	Create a task (modal: title, module, due date, estimate)
<code>/task-assign</code>	Founder	Assign task to a team member
<code>/task-start</code>	Anyone	Mark your assigned task as In Progress
<code>/task-update</code>	Anyone	Add a progress note to your task
<code>/task-review</code>	Anyone	Move your task to In Review
<code>/task-done</code>	Anyone	Mark task complete (slip check + reason if late)
<code>/task-cancel</code>	Founder	Cancel a task
<code>/task-block</code>	Anyone	Mark task blocked (triggers blocker workflow)
<code>/task-list</code>	Anyone	List tasks: yours / all / by module / by status
<code>/task-view</code>	Anyone	Full details of a specific task
<code>/my-tasks</code>	Anyone	Quick view of your active tasks
<code>/pickup</code>	Anyone	See and claim unassigned tasks

`cogs/modules.py` — Module Management

Command	Permission	Description
<code>/module-create</code>	Founder	Create a module (name, client, deadline)
<code>/module-list</code>	Anyone	List all active modules
<code>/module-status</code>	Anyone	All tasks in a module with status breakdown
<code>/module-deadline</code>	Founder	Update a module's deadline

`cogs/standups.py` — Daily Standups

Command	Permission	Description
<code>/standup</code>	Anyone	Submit today's standup (opens 3-question modal)
<code>/standup-view</code>	Founder	See someone's recent standups
<code>/standup-missed</code>	Founder	See who hasn't submitted today

Scheduled behavior: At 10am IST (weekdays), Jarvis DMs every active team member who hasn't submitted a standup yet with a reminder and a prompt to use `/standup`.

`cogs/blockers.py` — Blocker Tracking

Command	Permission	Description
<code>/blocker</code>	Anyone	Report a blocker (modal: describe it, which task, who's blocking you)
<code>/blocker-resolve</code>	Anyone	Resolve a blocker with a resolution note
<code>/blockers-open</code>	Anyone	See all currently open blockers
<code>/blockers-mine</code>	Anyone	See your own open blockers

`cogs/intelligence.py` — AI Commands + #ops-brain Listener

Command	Permission	Description
<code>/profile @person</code>	Founder	AI-generated team member profile from actual data
<code>/growth @person</code>	Founder	Performance trajectory over time
<code>/delivery-check</code>	Founder	Can we deliver [module] by [date]? Honest assessment.
<code>/context @person</code>	Anyone	Handoff context for a person (active tasks, dependencies, skills)
<code>/context-task</code>	Anyone	Full context and history for a specific task
<code>/bottlenecks</code>	Founder	Structural bottleneck analysis — recurring patterns

Passive behavior: Listens in `#ops-brain` channel. When Jarvis is @mentioned, compiles full context and responds via Claude API. Typing indicator shows while generating.

cogs/demo.py — Demo Readiness

Command	Permission	Description
<code>/demo-ready</code>	Anyone	Current demo readiness snapshot with traffic light
<code>/demo-add</code>	Founder	Add a feature to demo tracking
<code>/demo-update</code>	Founder	Update a feature's readiness status

cogs/admin.py — Admin + Setup

Command	Permission	Description
<code>/register @person [role]</code>	Founder	Register a team member in Jarvis
<code>/team</code>	Founder	See all registered team members and roles
<code>/context-add</code>	Founder	Manually inject context into Jarvis's memory
<code>/viewlog</code>	Admin	Current work log summary (migrated)
<code>/resetmonth</code>	Admin	Archive month and reset (migrated)
<code>/jarvis-status</code>	Founder	Health check — DB rows, last briefing, scheduler status

6. AI Engine (`ai_engine.py`)

6.1 The Context Architecture

When Jarvis needs to respond to anything, it compiles a context window from four tiers:

Tier 1 — Live State (always included, pulled fresh from DB):

- All registered team members + their current task status
- All open `blockers` (`with` hours open)
- Tasks due `in` the next `7` days
- Who is currently clocked `in`
- Velocity `this week`: tasks completed vs last week
- Who hasn't submitted standup today

Tier 2 — Recent Events (last 7 days from `context_log`):

- All context_log entries ordered newest first
- Capped at ~3,000 tokens if too long

Tier 3 — Historical Snapshots (from jarvis_snapshots):

- Weekly team snapshots for the past 4 weeks
- Person-specific snapshot if query is about a specific person
- Module snapshot if query is module-specific
- Capped at ~2,000 tokens

Tier 4 — Manual Context (from manual_context):

- Everything Jeevan/Subrat has manually injected
- Always fully included (usually short)

Total target: ~6,000 tokens of context. Then system prompt. Then query.

6.2 When AI Is Called

Trigger	Prompt file	Output destination
9am IST daily	morning_briefing.txt	DM to Subrat
@jarvis in #ops-brain	ops_brain.txt	Reply in channel
/profile @person	person_profile.txt	Ephemeral embed
/growth @person	person_profile.txt	Ephemeral embed
/delivery-check	delivery_prediction.txt	Ephemeral embed
/bottlenecks	ops_brain.txt	Ephemeral embed
Friday 6pm IST	weekly_narrative.txt	Post to #ops-brain
Clock-out with problem keywords	ops_brain.txt	DM to Subrat (flag only)

6.3 ai_engine.py Class Structure

```
from openai import AzureOpenAI
from datetime import datetime, timedelta, timezone
```

```

from config import config

class JarvisAI:
    def __init__(self, db):
        self.db = db
        # Microsoft Azure OpenAI – get key/endpoint/deployment
        # from Stuti
        self.client = AzureOpenAI(
            api_key=config.AZURE_OPENAI_API_KEY,
            api_version=config.AZURE_OPENAI_API_VERSION,
            azure_endpoint=config.AZURE_OPENAI_ENDPOINT,
        )
        self.model = config.AZURE_OPENAI_DEPLOYMENT # e.g.
        "gpt-4o" – strongest model available

        # — Context compilation —————

    def compile_context(self, focus_user_id=None, focus_module=None):
        """
        Compile the full tiered context string for AI calls.
        Returns a single string to put in the user message alongside the query.
        """
        parts = []
        parts.append(self._get_live_state())
        parts.append(self._get_recent_events(days=7))
        parts.append(self._get_snapshots(focus_user_id, focus_module))
        parts.append(self._get_manual_context())
        return "\n\n".join(filter(None, parts))

    def _get_live_state(self):
        # Query tasks, blockers, standup status, clock-in status

```

```

        # Format as readable summary
        ...

def _get_recent_events(self, days=7):
    # Query context_log for last N days
    # Return as list of content strings, newest first
    ...

def _get_snapshots(self, user_id=None, module=None):
    # Query jarvis_snapshots for relevant historical cont
ext
    ...

def _get_manual_context(self):
    # Query manual_context table
    ...

# — The log_event function — call this from EVERY cog —


---



def log_event(self, event_type, content, user_id=None,
              task_id=None, module_id=None, metadata=None):
    """
    Log a meaningful event to Jarvis's memory.
    content should be a plain English sentence — it goes
    directly to AI.
    """
    self.db.table("context_log").insert({
        "event_type": event_type,
        "user_id": user_id,
        "task_id": str(task_id) if task_id else None,
        "module_id": str(module_id) if module_id else None,
        "content": content,
        "metadata": metadata or {}
    })

```



```

    }).execute()

# — Scheduled AI calls —————

async def morning_briefing(self, subrat_user):
    """Generate and send the 9am briefing to Subrat."""
    ...

async def weekly_narrative(self, ops_brain_channel):
    """Generate and post the Friday evening narrative."""
    ...

# — On-demand AI calls —————

async def respond_to_query(self, query, channel_history=None):
    """Respond to a natural language query in #ops-brain."""
    ...

async def generate_profile(self, target_user_id):
    """Generate AI profile for a team member."""
    ...

async def delivery_check(self, module_id, target_date):
    """Assess: can we deliver this module by this date?"""
    ...

async def generate_growth(self, target_user_id):
    """Generate growth trajectory for a team member."""
    ...

# — Weekly snapshot generation —————

```

```

def generate_weekly_snapshot(self):
    """
    Compress the past week's context_log into a structured snapshot.
    Run Sunday midnight IST via scheduler.
    Saves to jarvis_snapshots so context doesn't grow unbounded.
    """
    . . .

```

6.4 The `log_event` Rule

This is the most important discipline in the whole codebase. Every cog, after every meaningful state change, must call `jarvis_ai.log_event()`. If it doesn't get logged, Jarvis doesn't know it happened.

Write `content` as something you'd say in a standup:

```

# GOOD – specific, numbers, names
jarvis_ai.log_event(
    event_type="task_complete",
    user_id=str(interaction.user.id),
    task_id=task["id"],
    content=f"Adhi completed 'API endpoint build' in 4.2h (estimated 3h, 40% over estimate). Not slipped.",
    metadata={"estimated_hours": 3, "actual_hours": 4.2, "slipped": False}
)

# BAD – vague, no numbers
jarvis_ai.log_event(event_type="task_complete", content="Task done.")

```

Things that **MUST** be logged:

- Task created, assigned, started, completed, cancelled, slipped
- Blocker opened, resolved, gone stale
- Standup submitted (one-line summary)
- Session ended (one-line summary from modal)
- Demo status updated
- Manual context added

6.5 Problem-Detection in Clock-Out Notes

After the `WorkDescriptionModal` saves a session, check the description for distress signals:

```
PROBLEM_KEYWORDS = [
    "stuck", "can't figure", "cannot figure", "haven't solve
d", "still not",
    "haven't figured", "not working", "confused", "lost 3 hou
rs", "lost 4 hours",
    "not making progress", "no idea", "completely blocked"
]

def is_quietly_struggling(description: str) -> bool:
    desc_lower = description.lower()
    return any(keyword in desc_lower for keyword in PROBLEM_K
EYWORDS)
```

If detected, post publicly in `#ops-brain`:

```
△ [Name] just clocked out – their session note suggests they
may be stuck:
"[first 150 chars of their description]"
Anyone free to pair with them?
```

This surfaces quietly struggling people to the whole team, not just founders — anyone can jump in to help.

7. Scheduler (`scheduler.py`)

```
from apscheduler.schedulers.asyncio import AsyncIOScheduler
from apscheduler.triggers.cron import CronTrigger
import pytz

IST = pytz.timezone('Asia/Kolkata')

def setup_scheduler(bot, jarvis_ai, config):
    scheduler = AsyncIOScheduler(timezone=IST)

    # 9:00 IST daily – Morning briefing to Subrat
    scheduler.add_job(
        morning_briefing_job,
        CronTrigger(hour=9, minute=0, timezone=IST),
        args=[bot, jarvis_ai, config]
    )

    # 10:00 IST Mon–Fri – Standup DM reminder (only to those
    who haven't submitted)
    scheduler.add_job(
        standup_reminder_job,
        CronTrigger(hour=10, minute=0, day_of_week='mon-fri',
        timezone=IST),
        args=[bot, config]
    )

    # 14:00 IST daily – Check for stale blockers (open > 24hrs, alert Subrat)
    scheduler.add_job(
        check_stale_blockers_job,
        CronTrigger(hour=14, minute=0, timezone=IST),
        args=[bot, jarvis_ai, config]
    )
```

```

# 18:00 IST Friday – Weekly narrative posted to #ops-brain
scheduler.add_job(
    weekly_narrative_job,
    CronTrigger(hour=18, minute=0, day_of_week='fri', timezone=IST),
    args=[bot, jarvis_ai, config]
)

# 00:00 IST Sunday – Compress week into jarvis_snapshots
scheduler.add_job(
    weekly_snapshot_job,
    CronTrigger(hour=0, minute=0, day_of_week='sun', timezone=IST),
    args=[jarvis_ai]
)

scheduler.start()
return scheduler

```

8. Event Flows

When someone runs `/task-done`

1. Look up task by ID (must be assigned to them)
2. Check if task is past due_date → if yes, set slipped=True, increment slip_count
3. If slipped: open a modal asking "What caused this to run late?" with a category dropdown
4. Save slip reason to slip_reasons table
5. Update task: status='done', completed_at=NOW()
6. Calculate actual hours:
 - Sum all completed_sessions.duration_seconds for this user since task.started_at
 - Convert to hours

- Store `in` `task_updates` as final note
- 7. Call `jarvis_ai.log_event("task_complete", content=f"[Name] completed '[title]' in X.Xh (estimated Yh). {'Slipped.' if slipped else ''})"`
- 8. Post to `#status` channel: `"{Name} ✅ completed '{title}'"`

When someone runs `/blocker`

1. Open modal: describe blocker / which task / who's blocking you
2. Save to blockers table (`status='open'`)
3. If `task_id` provided: update `tasks.status = 'blocked'`
4. Call `jarvis_ai.log_event("blocker_open", content=f"[Name] reported a blocker: '[description]'. Task: '[title]'. Blocking person: [if set].")`
5. Post to `#status` channel with blocker details
6. If `blocking_person_id` is set: DM that person with `"Hey, [Name] is waiting on you for: [description]"`
7. DM Subrat if blocker is on a critical task or blocking multiple people

Morning Briefing at 9am IST

1. Query DB for:
 - All open blockers + hours open
 - Tasks with `due_date = TODAY`
 - Tasks with `due_date = TODAY` that are not `'done'` or `'in_review'`
 - Velocity: tasks completed this week vs last week
 - Team hours logged yesterday (sum from `completed_sessions`)
 - People who haven't submitted standup in 2+ days
 - People who haven't clocked in in 2+ days
2. Compile context from all tiers (`ai_engine.compile_context`)

```
( )
```

3. Format data as structured text + send to Claude API with morning_briefing.txt prompt
4. Claude returns the formatted briefing text
5. DM Subrat with the briefing as a Discord embed
6. Log the briefing generation to context_log for continuity

@jarvis mentioned in #ops-brain

1. on_message listener fires (only in OPS_BRAIN_CHANNEL_ID)
2. Check if bot is mentioned → if not, ignore
3. Extract the query (strip the @jarvis mention)
4. Show typing indicator (async with channel.typing())
5. Compile full context (ai_engine.compile_context())
6. Send to Claude API with ops_brain.txt system prompt
7. Post response as a reply in the channel

9. main.py + config.py

main.py

```
import discord
from discord.ext import commands
import asyncio
from config import config
from database import db
from ai_engine import JarvisAI
from scheduler import setup_scheduler

intents = discord.Intents.default()
```

```

intents.message_content = True
intents.members = True

bot = commands.Bot(command_prefix="!", intents=intents)
jarvis_ai = JarvisAI(db)

COGS = [
    "cogs.clock",
    "cogs.stats",
    "cogs.tasks",
    "cogs.modules",
    "cogs.standups",
    "cogs.blockers",
    "cogs.intelligence",
    "cogs.demo",
    "cogs.admin",
]

@bot.event
async def on_ready():
    await bot.tree.sync()
    setup_scheduler(bot, jarvis_ai, config)
    asyncio.create_task(health_server())
    print(f"✅ Jarvis online as {bot.user}")

async def main():
    async with bot:
        for cog in COGS:
            await bot.load_extension(cog)
        await bot.start(config.TOKEN)

if __name__ == "__main__":
    asyncio.run(main())

```

config.py


```

import os
try:
    from dotenv import load_dotenv
    load_dotenv()
except ImportError:
    pass

class Config:
    # Discord
    TOKEN = os.environ.get("TOKEN")
    STATUS_CHANNEL_ID = int(os.environ.get("STATUS_CHAN
NEL_ID", 0))
    OPS_BRAIN_CHANNEL_ID = int(os.environ.get("OPS_BRAIN_C
HANNEL_ID", 0))
    STANDUP_CHANNEL_ID = int(os.environ.get("STANDUP_CHA
NNEL_ID", 0))
    ADMIN_IDS = [x.strip() for x in os.environ.
get("ADMIN_IDS", "").split(",") if x.strip()]
    FOUNDER_IDS = [x.strip() for x in os.environ.
get("FOUNDER_IDS", "").split(",") if x.strip()]
    MORNING_BRIEFING_USER_IDS = [x.strip() for x in os.enviro
n.get("MORNING_BRIEFING_USER_IDS", "").split(",") if x.strip
()]
    # Both Jeevan and Subrat receive the morning briefing

    # Supabase
    SUPABASE_URL = os.environ.get("SUPABASE_URL")
    SUPABASE_KEY = os.environ.get("SUPABASE_KEY")

    # AI – Microsoft Azure OpenAI (get these from Stuti)
    AZURE_OPENAI_API_KEY = os.environ.get("AZURE_OPENAI_A
PI_KEY")
    AZURE_OPENAI_ENDPOINT = os.environ.get("AZURE_OPENAI_E
NDPOINT")
    AZURE_OPENAI_DEPLOYMENT = os.environ.get("AZURE_OPENAI_D

```

```

EMPLOYMENT", "gpt-4o")
    AZURE_OPENAI_API_VERSION = os.environ.get("AZURE_OPENAI_A
PI_VERSION", "2024-08-01-preview")

    # Server
    PORT = int(os.environ.get("PORT", 8080))

config = Config()

```

10. `utils.py` — Shared Utilities

```

from datetime import datetime, timezone, timedelta
from config import config
import discord

IST_OFFSET = timedelta(hours=5, minutes=30)

def to_ist(dt):
    if dt.tzinfo is None:
        dt = dt.replace(tzinfo=timezone.utc)
    return dt + IST_OFFSET

def now_ist():
    return datetime.now(timezone.utc) + IST_OFFSET

def format_duration(total_seconds):
    total_seconds = int(total_seconds)
    h = total_seconds // 3600
    m = (total_seconds % 3600) // 60
    s = total_seconds % 60
    return f"{h}h {m}m {s}s"

def get_month_key(dt=None):
    if dt is None:
        dt = datetime.now(timezone.utc)

```

```

    return dt.strftime("%Y-%m")

def is_founder(user_id: str) -> bool:
    return str(user_id) in config.FOUNDER_IDS

def is_admin(user_id: str) -> bool:
    return str(user_id) in config.ADMIN_IDS

def require_founder():
    """Decorator-style check for slash commands."""
    async def predicate(interaction: discord.Interaction) -> bool:
        if not is_founder(str(interaction.user.id)):
            await interaction.response.send_message(
                "❌ This command is for founders only.", ephemeral=True
            )
            return False
        return True
    return predicate

```

11. Environment Variables (Full List)

```

# Discord
TOKEN=your_discord_bot_token
STATUS_CHANNEL_ID=123456789          # Existing clock-in/out channel
OPS_BRAIN_CHANNEL_ID=123456789       # The #ops-brain channel
STANDUP_CHANNEL_ID=123456789         # Where standup summaries post
ADMIN_IDS=discord_id_1,discord_id_2 # Jeevan's and Subrat's Discord user IDs (for destructive commands)
FOUNDER_IDS=discord_id_1,discord_id_2 # Same – can overlap with ADMIN_IDS
MORNING_BRIEFING_USER_IDS=discord_id_jeevan,discord_id_subrat

```

```
# Both founders get the briefing

# Supabase (same project as existing bot)
SUPABASE_URL=https://xxx.supabase.co
SUPABASE_KEY=your_supabase_anon_key

# AI – Microsoft Azure OpenAI (ask Stuti for these)
AZURE_OPENAI_API_KEY=your_azure_openai_key
AZURE_OPENAI_ENDPOINT=https://your-resource.openai.azure.com/
AZURE_OPENAI_DEPLOYMENT=gpt-4o # strongest model a
available in your region
AZURE_OPENAI_API_VERSION=2024-08-01-preview

# Server
PORT=8080
```

12. System Prompts (Starting Versions)

These are starting points. Tune them after seeing 2–3 weeks of real output.

prompts/morning_briefing.txt

You are Jarvis, the AI operating brain for Nexrova – a 2-founder, 6-intern startup building hospitality tech. You generate the daily morning briefing for Subrat, the team lead.

Your briefing is:

- Direct and specific – use names, numbers, task titles, hours
- Never padded or vague – every sentence is actionable
- Structured in exactly this order: Company State → What Needs Attention → Recommended First Action
- Under 300 words total

The person you're briefing (Subrat) has limited time. He'll r

Read **this in** under **60** seconds before starting his day.
Surface only the top **2–3** things that actually need him today.
Not everything. Just the most urgent.

If there are no blockers, say so clearly. If velocity is fine, don't pad **with** a mention **of** it.
Your recommended first action should be **ONE** specific thing, not a list.

prompts/ops_brain.txt

You are Jarvis, the **AI** operating brain **for** Nexrova – a **2**-founder startup building hospitality technology.
You are responding to a message **in** the #ops-brain channel from Jeevan or **Subrat** (the founders).

You have full context about the team: tasks, blockers, velocity, session history, standups.
Use that data **in** your responses. Be specific – actual names, task titles, hours, counts.

Rules:

- Answer directly. No preambles like "Great question!"
- Under **150** words unless a detailed breakdown is genuinely required
- If you don't **have data for something**, say "I don't have data on that" – never guess or invent
- If the question is about delivery, use actual velocity numbers
- If the question is about a person, use their actual task history

prompts/weekly_narrative.txt

You are Jarvis, the **AI** operating brain **for** Nexrova. Every Friday you write a brief narrative summary **of** the week.

This narrative:

- Is **ONE** paragraph, **100–150** words
- Reads like a well-written ops update – not a task list
- Highlights the most significant thing shipped, the biggest risk going into next week, and velocity direction
- Is specific: mentions module names, task names, people's names where relevant
- Can be pasted directly into a client update or investor message

Do not use bullet points. Write **in** flowing prose. Past tense **for** what happened, present/future tense **for** risks and next week.

prompts/person_profile.txt

You are Jarvis, the **AI** operating brain **for** Nexrova. Generate a factual profile **of** a team member based on the data provided.

Structure your profile **as** four sections:

STRENGTHS – based on task types completed quickly, on-time rate, pickup behavior

WATCH AREAS – where they slip, underestimate, or **struggle** (be honest, not cruel)

WORKING PATTERNS – session timing, average length, check-**in** needs **if** any

CURRENT STATE – active tasks, output level **this** week, open blockers

Rules:

- Every claim must be grounded **in** the data. Say "based on 5 completed tasks, early signs suggest..." **if** data is thin
- No intro sentence. Start directly **with STRENGTHS**
- Plain text. No markdown. No bullet points inside sections – write **in** short prose
- This profile is **for** a co-founder making real decisions about task assignment

prompts/delivery_prediction.txt

You are Jarvis, the **AI** operating brain **for** Nexrova. Assess whether the team can deliver a module by a given date.

Base your assessment entirely on:

- Open tasks remaining **in** the module and their complexity
- Historical velocity **for this module** (tasks/week)
- Available people assigned to **this** module
- Current blockers that could slow things

Structure your response **as**:

1. Honest **verdict** (Likely / Unlikely / Possible **if** X)
2. The **math** (tasks remaining, completion rate, days available)
3. What would need to change to make the date **realistic** (**if** unlikely)
4. The realistic date **if** the target date won't work

Be direct. Don't soften the assessment. The founder asking **this** is making a client commitment.

13. requirements.txt

```
discord.py>=2.3.0
supabase>=2.0.0
```

```
openai>=1.40.0
APScheduler>=3.10.4
pytz
matplotlib
python-dotenv
```

14. Phased Implementation Plan

Work through these phases in order. Do not skip ahead — later phases depend on earlier ones.

PHASE 0: Setup & Migration (Day 1)

Goal: The existing bot works from the new structure. Nothing breaks.

- ☐ Create the full folder structure above
- ☐ Copy existing `main.py` content into the new structure
- ☐ Move clock-in/out commands into `cogs/clock.py` as a Cog class
- ☐ Move stats commands into `cogs/stats.py`
- ☐ Create `config.py` and replace all `os.environ.get()` calls to use it
- ☐ Create empty `cogs/__init__.py`
- ☐ Run the full DB schema SQL in Supabase (all new tables)
- ☐ Set up `.env` with all variables
- ☐ Test: `/clockin`, `/clockout`, `/teamstatus`, `/serverreport` all work

Done when: Bot is online and all migrated commands work identically to before.

PHASE 1: Task System (Days 2–5)

Goal: Full task lifecycle works end to end.

Day 2 — Registration + Modules

- ☐ Build `/register @person [role]` in `admin.py` (upsert into `team_members`, confirm with embed)

☐ Build `/module-create` in `modules.py` (modal: name, client name, deadline; insert into `modules`)

☐ Build `/module-list`

Day 3 — Core Task Commands

☐ Build `/task-create` in `tasks.py` (modal: title, description (optional), module (select), due date, estimate hours; insert into `tasks`)

☐ Build `/task-assign @person [task-id]`

☐ Build `/task-list` with filter options

Day 4 — Task Lifecycle

☐ Build `/task-start [task-id]` (check caller is the assignee; update status to 'in_progress', set started_at)

☐ Build `/task-done [task-id]` (check if past due_date → if yes, open slip reason modal; update task, log event)

☐ Build `/pickup` (query unassigned tasks; show paginated list with select menu; assign selected task to caller)

Day 5 — Supporting Commands

☐ `/task-update [task-id]` — progress note modal

☐ `/task-view [task-id]` — full task embed

☐ `/task-block [task-id]` — mark blocked, prompt to file blocker

☐ `/my-tasks` — list caller's in-progress tasks

☐ `/module-status [module]` — tasks in module by status

Done when: Founders can create tasks, assign to interns, interns can start/update/complete them, slippage is tracked.

PHASE 2: Daily Operations (Days 6–9)

Goal: Standups, blockers, and demo readiness all working.

Day 6 — Standup Command

- ☐ Build `/standup` in `standups.py` (modal: "What did you work on?", "What's your focus today?", "Any blockers or risks?"; save to `standups` table; post summary to `#standup` channel)
- ☐ Build `/standup-missed` (admin, lists who hasn't submitted today)
- ☐ Build `/standup-view @person` (last 5 standups as embeds)

Day 7 — Standup Scheduler

- ☐ Set up APScheduler in `scheduler.py`
- ☐ 10am IST weekday job: DM everyone in `team_members` who hasn't submitted today's standup
- ☐ Test by temporarily setting it to 1 minute from now

Day 8 — Blocker System

- ☐ Build `/blocker` in `blockers.py` (modal: describe, which task (optional), who's blocking (optional); insert to `blockers`, update task if task_id given; DM blocking person if set)
- ☐ Build `/blocker-resolve [id]` (modal: how was it resolved?; update blocker record, calculate hours_open)
- ☐ Build `/blockers-open` and `/blockers-mine`

Day 9 — Demo Readiness

- ☐ Build `/demo-add` in `demo.py` (modal: feature name, description, link to module; insert to `demo_features`)
- ☐ Build `/demo-update [feature]` (select: ready / partial / not_ready; text input: status note)
- ☐ Build `/demo-ready` (query all `demo_features`; calculate %: $(ready \ 1 + \ partial \ 0.5) / total * 100$; show traffic light per feature; show overall score; show recommendation)

Done when: Standups DM at 10am, blockers tracked with DM notifications, demo readiness is visible with one command.

PHASE 3: Context Accumulation (Days 10–11)

Goal: Every meaningful event in the system writes to context_log. Nothing falls through.

Day 10 — Wire Up All Events

Go through every cog and add `jarvis_ai.log_event()` after every state change:

clock.py:

- After WorkDescriptionModal saves: `log_event("session_end", content=f"{name} worked Xh: [first 80 chars of description]")`

tasks.py:

- `/task-create` → `log_event("task_create", ...)`
- `/task-assign` → `log_event("task_assign", ...)`
- `/task-start` → `log_event("task_start", ...)`
- `/task-done` → `log_event("task_complete", ...)` ← already planned
- `/task-done (slipped)` → `log_event("task_slip", ...)`
- `/pickup` → `log_event("task_pickup", ...)`
- `/task-block` → `log_event("task_block", ...)`

standups.py:

- After `/standup` saves → `log_event("standup", content=one-line summary of all 3 answers)`

blockers.py:

- `/blocker` → `log_event("blocker_open", ...)`
- `/blocker-resolve` → `log_event("blocker_resolve", content=f"[name] resolved blocker '{description}' after {hours:.1f}h. Resolution: [note]")`

demo.py:

- `/demo-update` → `log_event("demo_update", content=f"[name] updated '{feature}' to {status}: [note]")`

Day 11 — Handoff Commands + Manual Context

- ☐ Build `/context-add` in `admin.py` (modal: content, type (client/decision/risk/general); insert to `manual_context`; log to context_log as well)
- ☐ Build `/context @person` in `intelligence.py` (query: active tasks, recent sessions, open blockers, completed task titles (skills proxy); format as rich embed)
- ☐ Build `/context-task [task-id]` (all task_updates, status history, who's worked on it, blockers linked to it)

Done when: Every event in the system is in context_log. Query the table and it should read like a team activity log.

PHASE 4: AI Layer (Days 12–16)

Goal: All AI features working. Morning briefing DMs Subrat at 9am.

Day 12 — AI Engine Setup

- ☐ Install `openai` library and configure the Azure OpenAI client (key/endpoint/deployment from Stuti)
- ☐ Build `JarvisAI` class skeleton in `ai_engine.py`
- ☐ Implement `compile_context()` function (run it manually and print the output — check it's readable and under 6,000 tokens; tune truncation logic until it's good)
- ☐ Implement `log_event()` function

Day 13 — Morning Briefing

- ☐ Write `prompts/morning_briefing.txt`
- ☐ Implement `morning_briefing()` function (compile live state; call Claude API with the prompt; return formatted text)
- ☐ Add to scheduler at 9am IST
- ☐ Test by calling it manually
- ☐ Fix and tune prompt until output matches the example in this doc

Day 14 — #ops-brain Listener

- ☐ In `intelligence.py`, add `@commands.Cog.listener()` for `on_message` (check: channel == OPS_BRAIN_CHANNEL_ID AND bot.user in message.mentions; extract query;

show typing; call `jarvis_ai.respond_to_query(query)` ; public reply visible to whole team)

- ☐ Write `prompts/ops_brain.txt`
- ☐ Test with: "@jarvis who's had the best week?"

Day 15 — Profile + Growth

- ☐ Write `prompts/person_profile.txt`
- ☐ Implement `generate_profile(user_id)` in JarvisAI (pull all tasks by this person, completion rate, estimate accuracy, session patterns, blockers; focus context on this person; call Claude)
- ☐ Build `/profile @person` command in `intelligence.py`
- ☐ Implement `generate_growth(user_id)` — same but sliced by week
- ☐ Build `/growth @person` command

Day 16 — Delivery Check + Bottlenecks

- ☐ Write `prompts/delivery_prediction.txt`
- ☐ Implement `delivery_check(module_id, target_date)` (pull open tasks, completed tasks (velocity), available assignees, open blockers; calculate tasks/week, working days until target_date, projected completion; send to Claude with the math pre-computed)
- ☐ Build `/delivery-check` command
- ☐ Build `/bottlenecks` — query slip_reasons categories + blocker reporter_ids, send to ops_brain prompt

Done when: All AI commands work. Morning briefing posts to Subrat's DMs at 9am.

PHASE 5: Weekly Intelligence (Days 17–19)

Goal: Friday narrative, weekly snapshots, session flags.

Day 17 — Weekly Narrative

- ☐ Write `prompts/weekly_narrative.txt`

- ☐ Implement `weekly_narrative(channel)` in JarvisAI (pull tasks completed this week, velocity vs last week, blockers opened/resolved, demo status; send to Claude; post result to #ops-brain)
- ☐ Add to scheduler: Friday 6pm IST
- ☐ Test manually first

Day 18 — Weekly Snapshots

- ☐ Implement `generate_weekly_snapshot()` in JarvisAI (for each team member compute week's stats; compress into a structured summary, save to `jarvis_snapshots`; for each module save status snapshot; add Sunday midnight IST job to scheduler)
- ☐ Update `compile_context()` to use these snapshots for weeks older than 7 days

Day 19 — Session Intelligence Flags

- ☐ In `clock.py`, after `WorkDescriptionModal.on_submit` saves the session: run `is_quietly_struggling(description)` check; if True post publicly to `#ops-brain` with the flag message
- ☐ Test with a session description containing "stuck" or "can't figure it out"

Done when: Jarvis posts Friday narratives, snapshots keep context manageable, session flags catch quiet struggles.

PHASE 6: Polish (Days 20–25)

- ☐ Add error handling everywhere — every DB call in a try/except
- ☐ Handle Discord edge cases: user not found, channel deleted, DM failed (user has DMs closed)
- ☐ Add `/jarvis-status` health check command (scheduler running? last briefing? DB row counts?)
- ☐ Tune AI prompts based on 1–2 weeks of real output
- ☐ Add rate limiting to the #ops-brain listener (max 1 query per 30 seconds)
- ☐ Write a `README.md` for the next person who touches this codebase

15. Key Design Decisions (Understand These)

Why task management is IN Jarvis, not Notion/Linear

All of Jarvis's analytics require task data. External tools need API integrations that can fail, drift, or go stale. Keeping everything in Supabase means Jarvis always has fresh, complete, real-time data. The slight inconvenience of managing tasks in Discord pays off in data fidelity.

Why context_log is plain English, not JSON

The content field goes directly into the AI prompt. Human-readable sentences ("Adhi completed X in Y hours, 40% over estimate") work significantly better in AI context than structured JSON blobs. The `metadata` JSONB field is for analytics queries — the `content` field is for AI.

Why profiles are generated per-request, not cached

The team is 6 people. Generating a profile takes 2–3 seconds and costs <\$0.05. Caching risks stale insights. Add caching only if the team scales to 15+ people.

Why the AI doesn't calculate, it interprets

For delivery predictions, compute the math in Python (tasks remaining / velocity / days). Send Claude the computed numbers with the question "what does this mean?" This is more reliable and cheaper than asking Claude to do arithmetic.

Why there are separate ADMIN_IDS and FOUNDER_IDS

Founders (Jeevan + Subrat) can see all team data and create/manage tasks. Admins (probably just them for now) can do destructive things like `resetmonth`. Separating them means you can grant someone admin for archiving without giving them full founder access, if that ever comes up.

16. Questions for Jeevan/Subrat

These don't block building — but answers will refine specific decisions.

For task system:

1. Can interns create tasks themselves, or is that founder-only? (Current assumption: founder-only) even interns can create tasks for them

2. Can interns see each other's task details, or just their own? (Current assumption: anyone can see all tasks) yeah they can see anyone

For standups:

1. Are the 3 standup questions exactly "Yesterday / Today / Blockers?" — or different?
2. What time IST should the standup reminder go out? (Current assumption: 10am)

For AI:

1. Should the morning briefing go to Subrat only, or both you and Subrat?
2. Should the @jarvis listener in #ops-brain reply publicly (visible to team) or only privately to founders? (Current assumption: public replies in the channel)